When Agentic Glue melts

EXPLOITING CLOUDFLARE CODE MODE & WORKERS

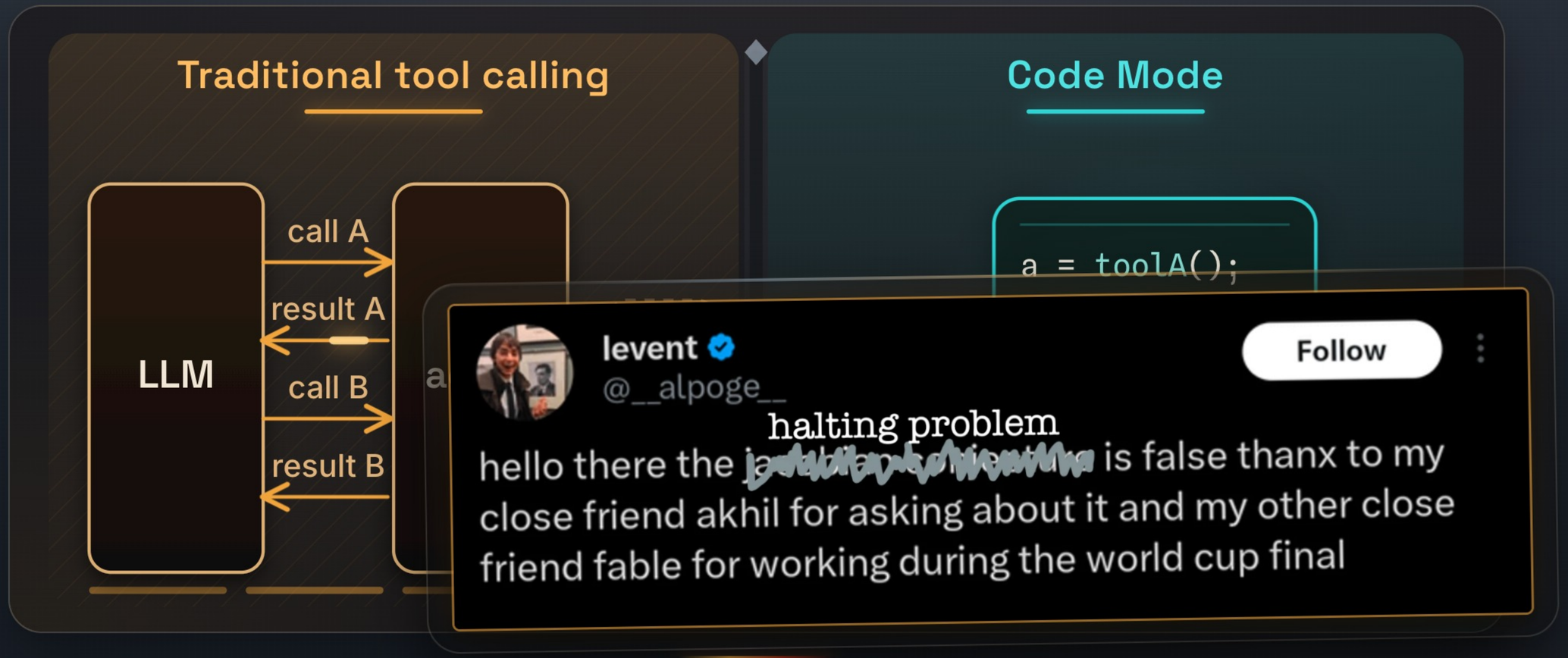
CHECK POINT FORMERLY CYATA

YARDEN PORAT · SHAHAR TAL ·

black hat

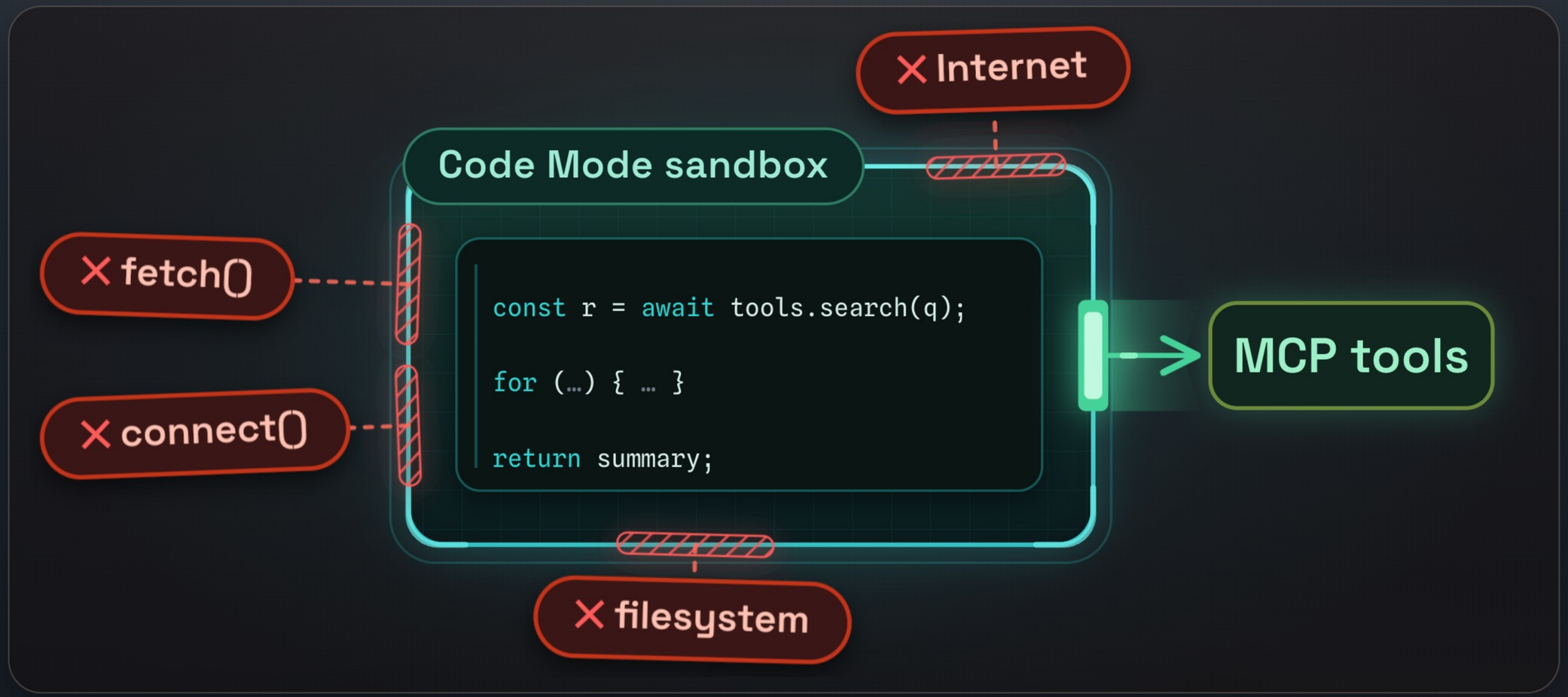
Guardrails: a checkpoint on every tool call

Code Mode: LLMs don't like round-trips



But now the model writes the code - the checkpoint still sees the calls, and **nothing of the intent.**

The sandbox: MCP tools in, nothing else out



01

We set out to research Code Mode

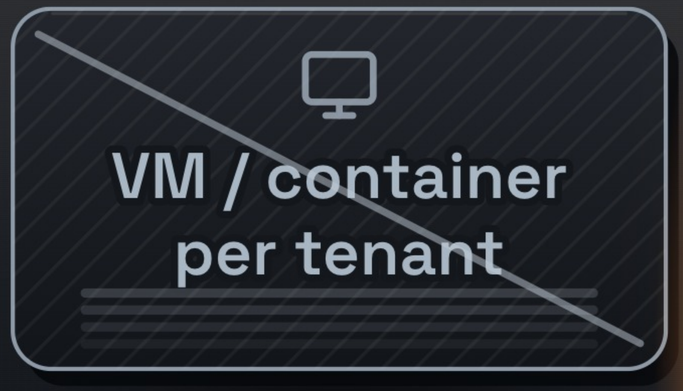
where does the model's code actually run?

Under Code Mode: workerd

Workers runs your code at the edge, on every request. workerd is the runtime underneath it.

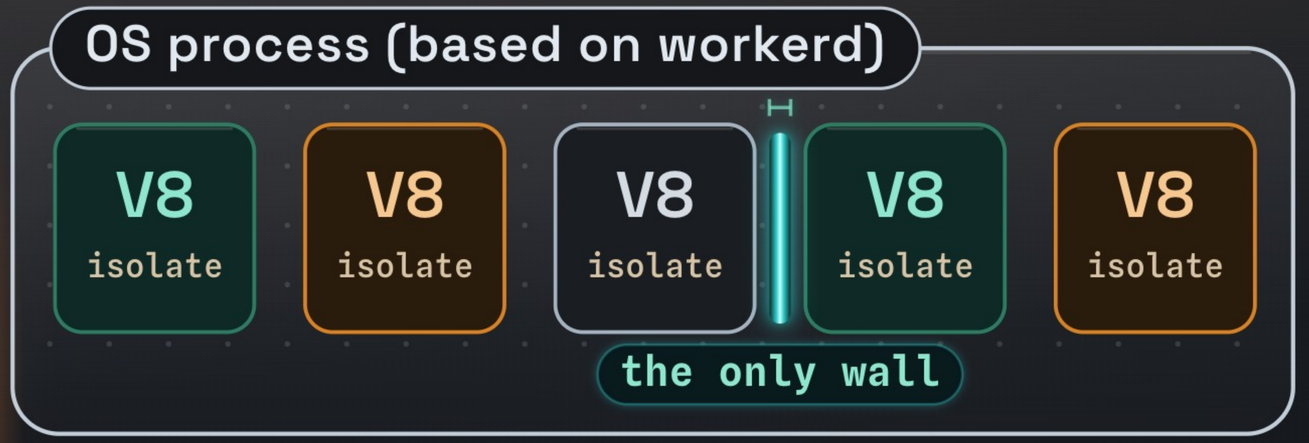
Why isolates, not containers

Many customers, one machine, on demand for every request. A container each is far too heavy for that.



10s–100s of ms

cold start, plus a real memory footprint each



single-digit ms

~100× faster to start · many per process

one process

IN-PROCESS sandboxing is the hard part

Why audit the runtime, not the seam

Like setting out to break an AI coding assistant - and going to audit Docker's source instead.

1 — **A bold in-process bet**

Tenant separation on **one software boundary.**

2 — **No public scrutiny**

`workerd`: **two** advisories, both Moderate.

3 — **A huge native surface**

Custom **C++**, reachable from untrusted JS.

4 — **Blast radius**

Not just Code Mode. **All of Workers.**

5 — **AI security has a low-level side** | The layers **under the prompts.**

How big is “all of Workers”?

5.5M+

CLOUDFLARE DEVELOPERS (Q1 FY2026)

+1M in one quarter

hundreds of billions

AGENTIC REQUESTS / MONTH

“growing exponentially”

>10%

NETWORK REQUESTS USE WORKERS

** July 2020. Cloudflare hasn’t restated a Workers traffic share since.*

A bug here is not contained to a feature.

They knew V8 would break

** scope: V8's memory.*

02

Vulnerabilities

where memory-unsafe glue meets attacker-written JS

The attack surface

01

V8

JIT · parser · GC · ArrayBuffer

THE ENGINE · CAGE-GUARDED

02

Node replacements

node:zlib · node:crypto · node:buffer · node:fs

03

Web APIs, natively implemented

URLPattern · HTMLRewriter · CompressionStream · crypto.subtle

JSG · the glue

1 2 3 Most of the code is C++ , not JavaScript.

2 *“Node.js trusts the application code that uses its APIs.”*

Five bugs. Two deep dives.

5

vulnerabilities in `workerd`

2

rated **Critical** by Cloudflare

2

we'll dig into **today**

URLPattern

node:zlib

URLPattern, the router

Web-standard API

URLPattern · the happy path

```
> const p = new URLPattern({ pathname: '/blog/:year/:month/:slug' });
> p.exec('https://example.com/blog/2025/03/post').pathname.groups;

→ { year: '2025', month: '03', slug: 'post' }
```

Actually, it's a regex

```
/blog/:year/:month/:slug
```

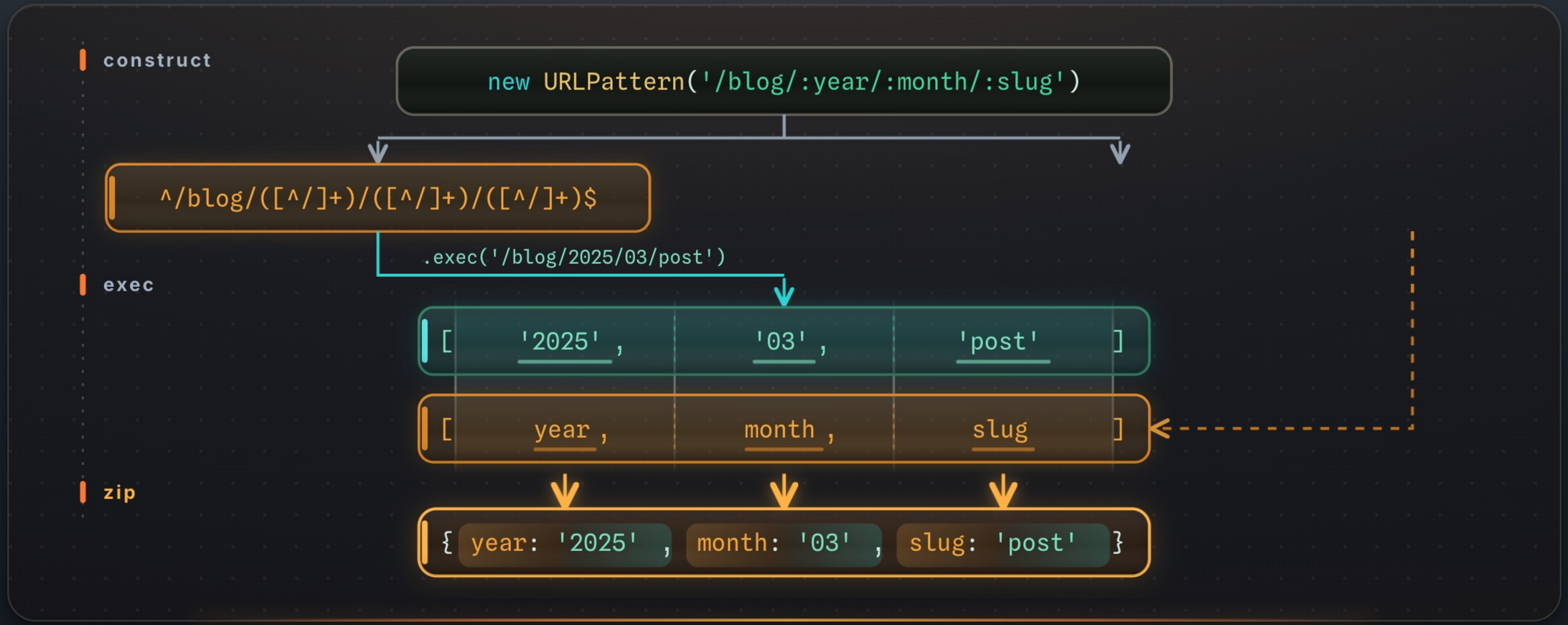
compiles to

```
^/blog/([^/]+)/([^/]+)/([^/]+)$
```

`/blog/:year/:month/:slug`	`year` `month` `slug`
`/blog/:year/:month/(\d+)`	`year` `month` `0`

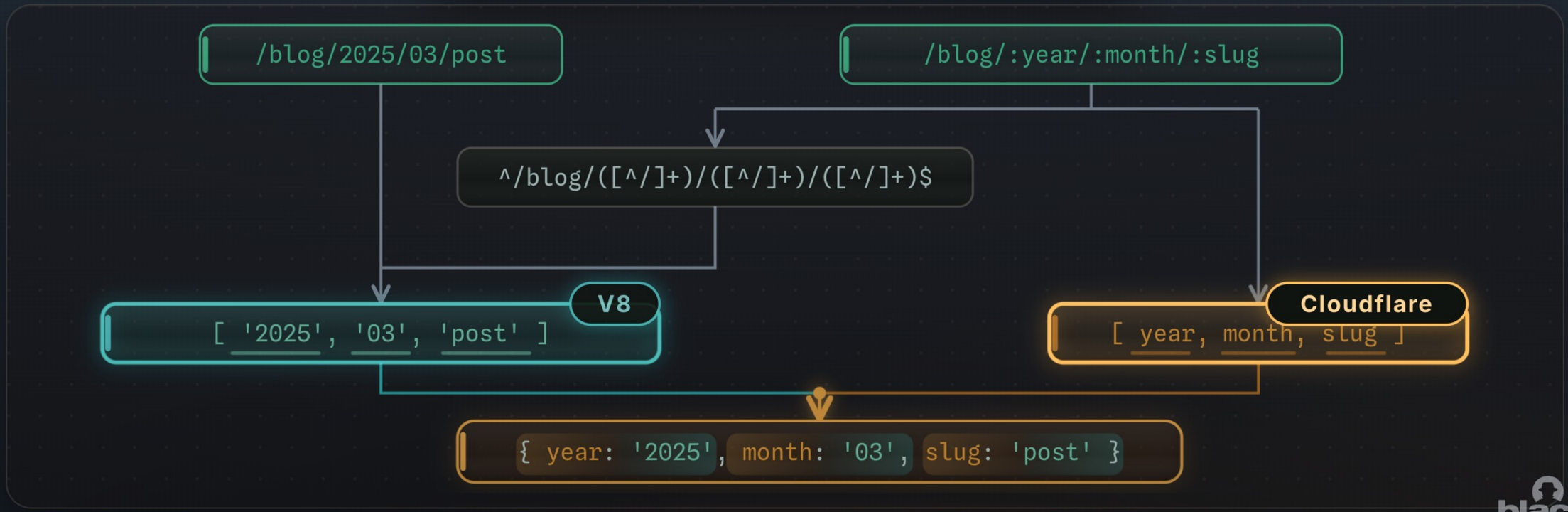
raw regex is allowed - and an **unnamed** group has no name to give, so it gets an **index**

Construct, then exec

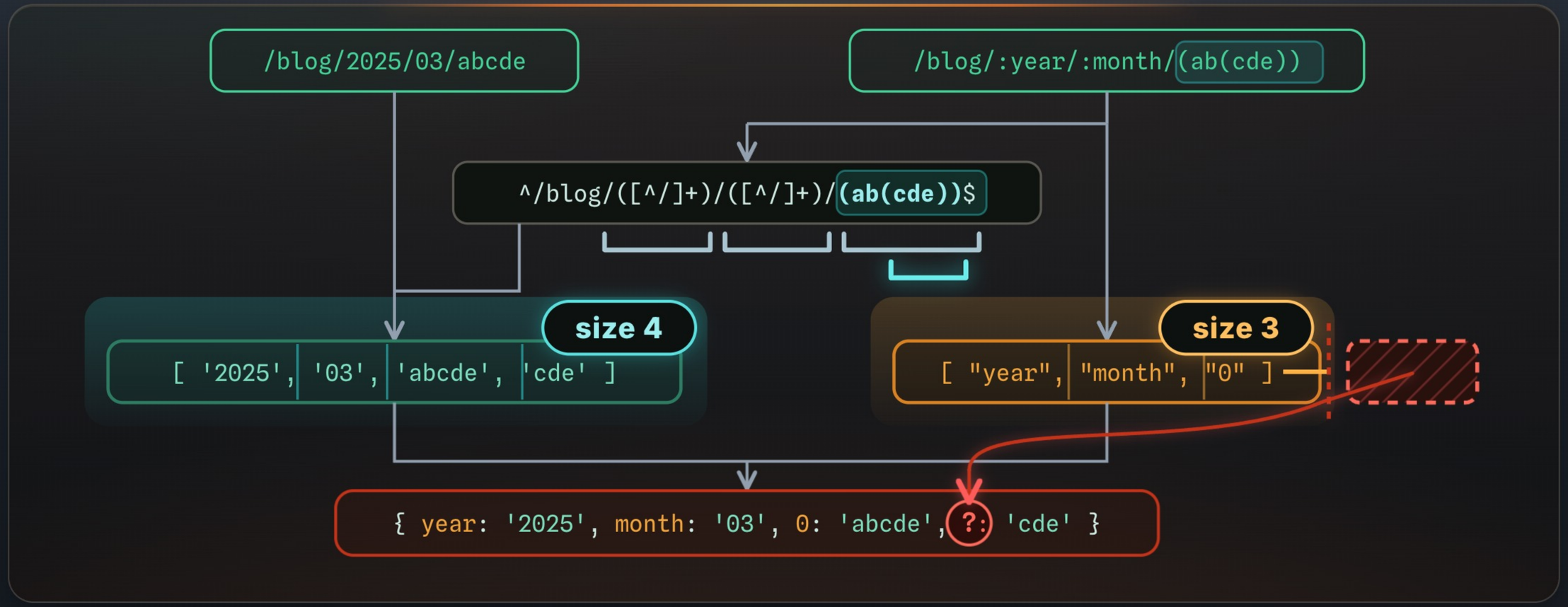


Two counts, wrong check

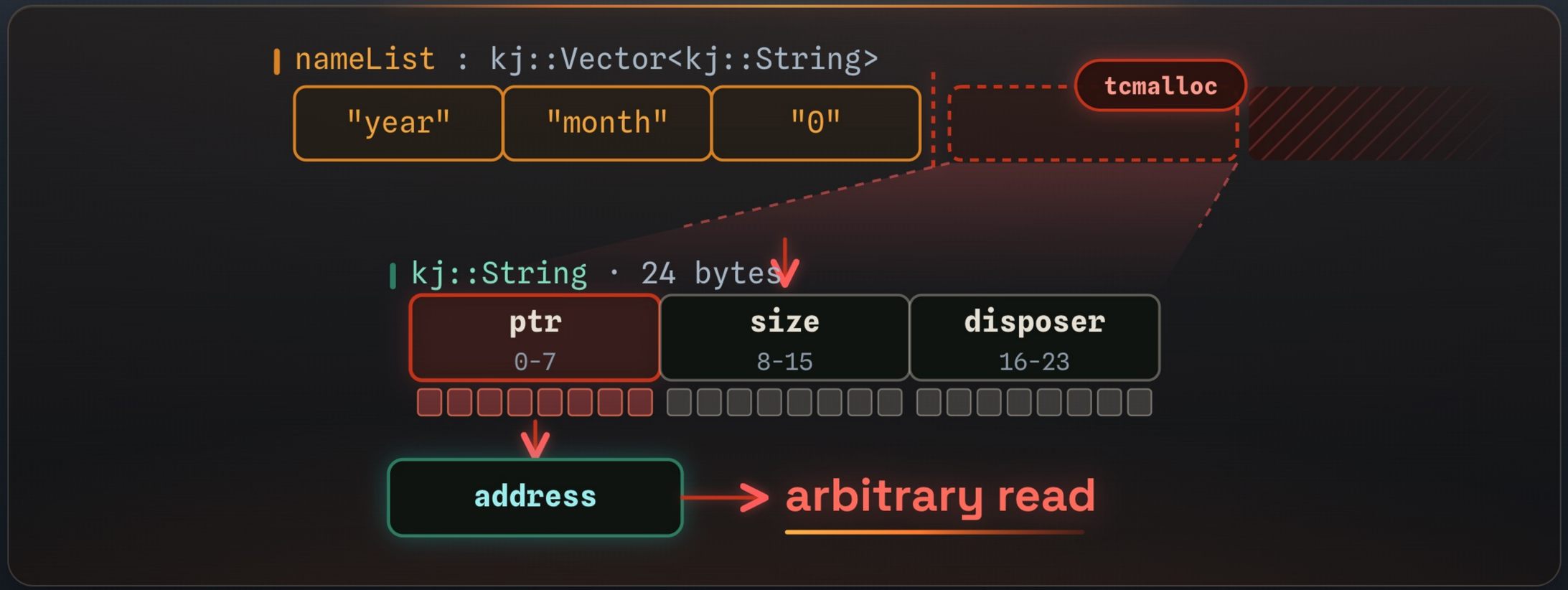
```
uint32_t length = array.size();
uint32_t index  = 0;

while (index < length) {
    groups.add({ name: nameList[index] });
    ++index;
}
```



The mismatch - nesting groups



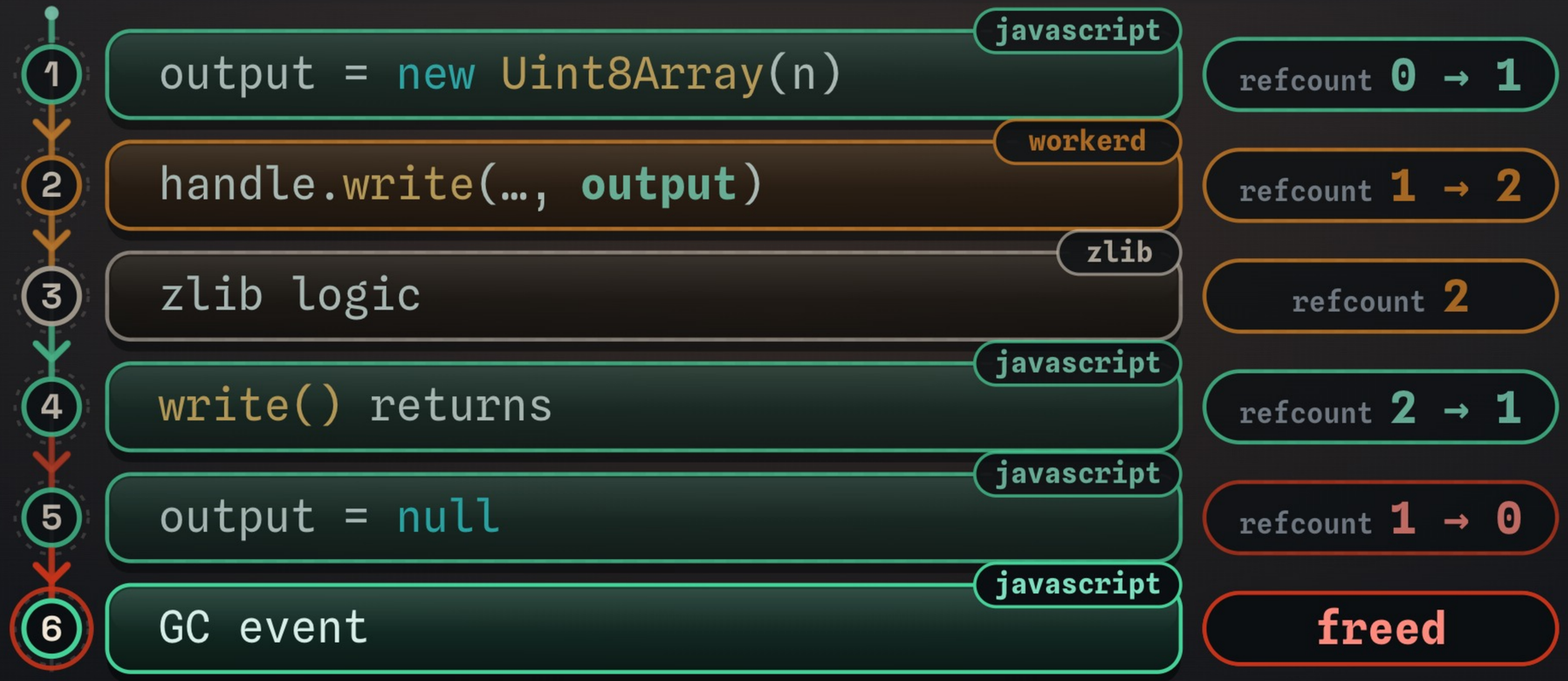
The primitive



node:zlib

```
const handle = new DeflateRaw();
const input  = Buffer.from("hello world");
const output = Buffer.alloc(64);
handle.write(input, output);
```

Lifetime of output



the glue forgets

z_stream zlib's internal state struct

```
1  void setBuffers(ArrayPtr<byte> input, ArrayPtr<byte> output) {
2    stream.next_in  = input.begin();   // raw ptr → JS input buffer
3    stream.next_out = output.begin();  // raw ptr → JS output buffer
4  }
```

~~stream.next_out = nullptr;~~ **MISSING**

Finding the Use in Use-After-Free

`handle.stream.next_out → dangling ptr`

IDEA 1

Call `write()` again

```
handle.write(input, output2)
```

↓

```
next_out = output2
```

IDEA 2

Call `params()`

```
handle.params(level, strategy)
```

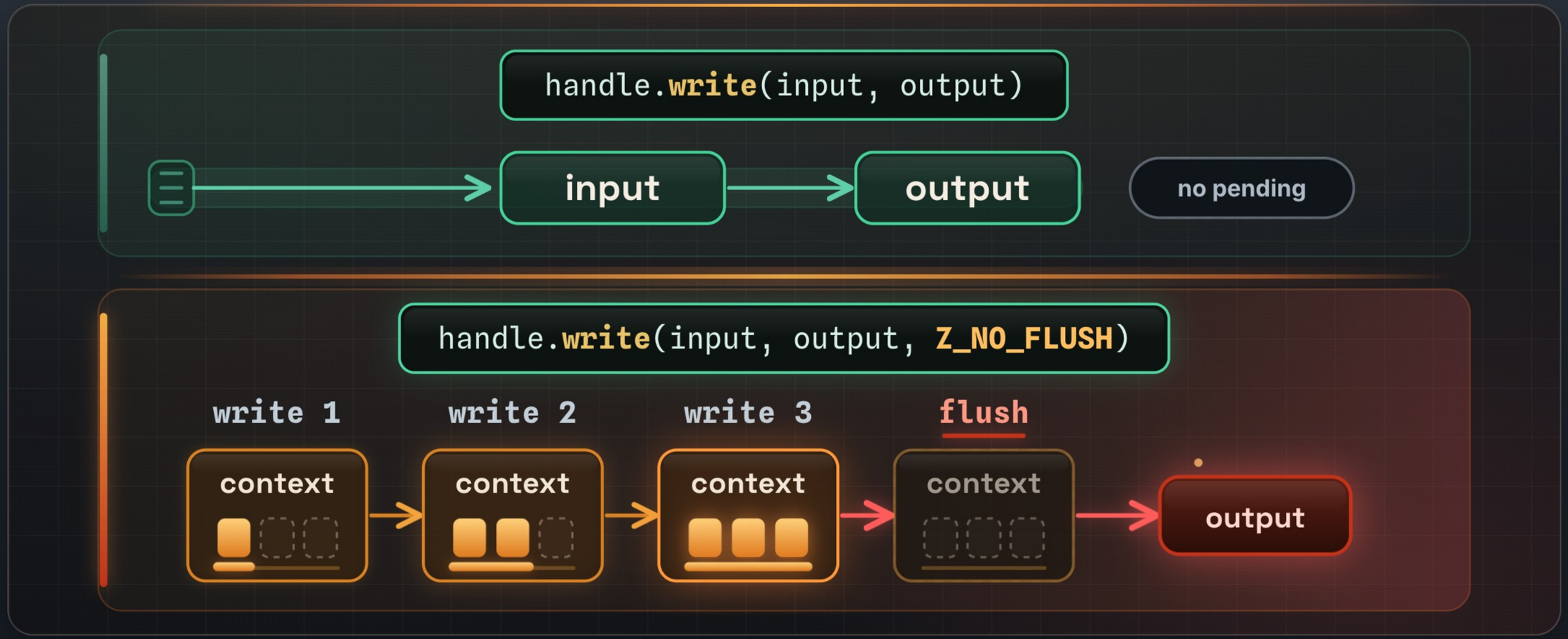
↓

```
deflateParams(&stream, …)
```

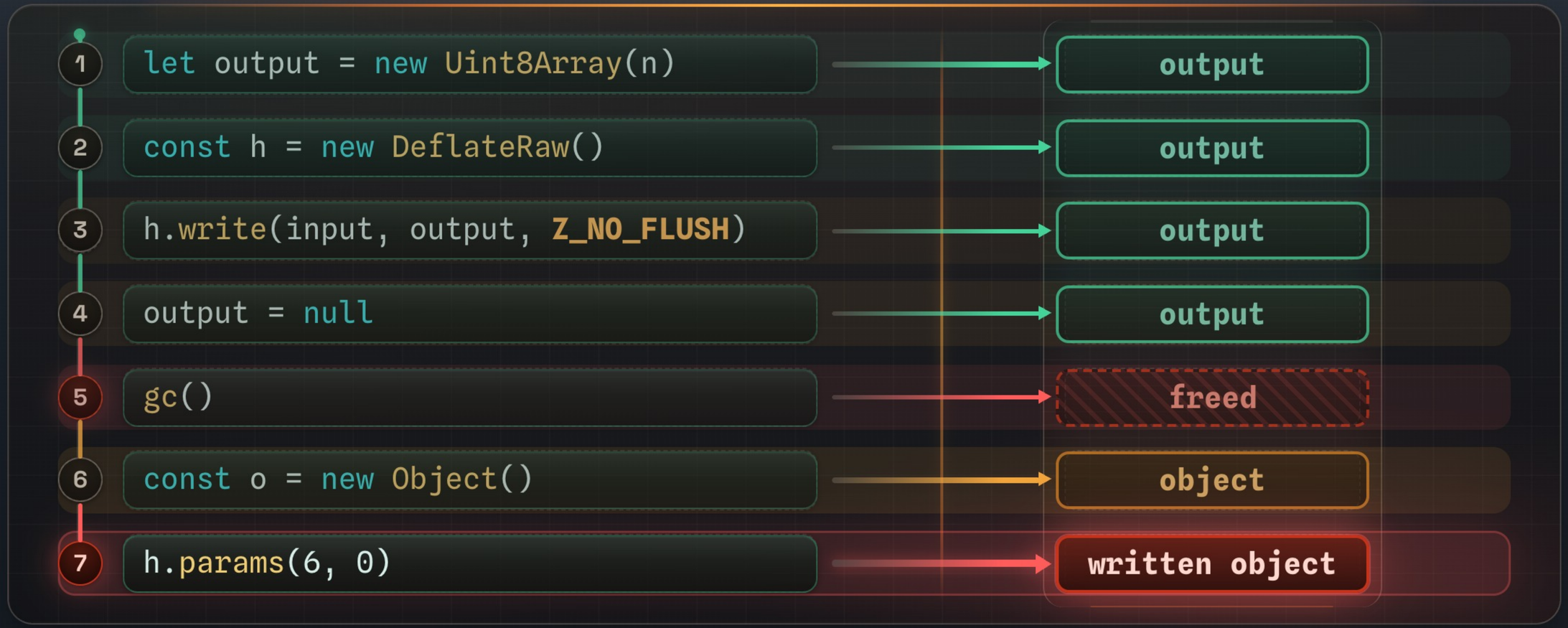
↓

flushing remaining input

zlib flushing



The full use-after-free



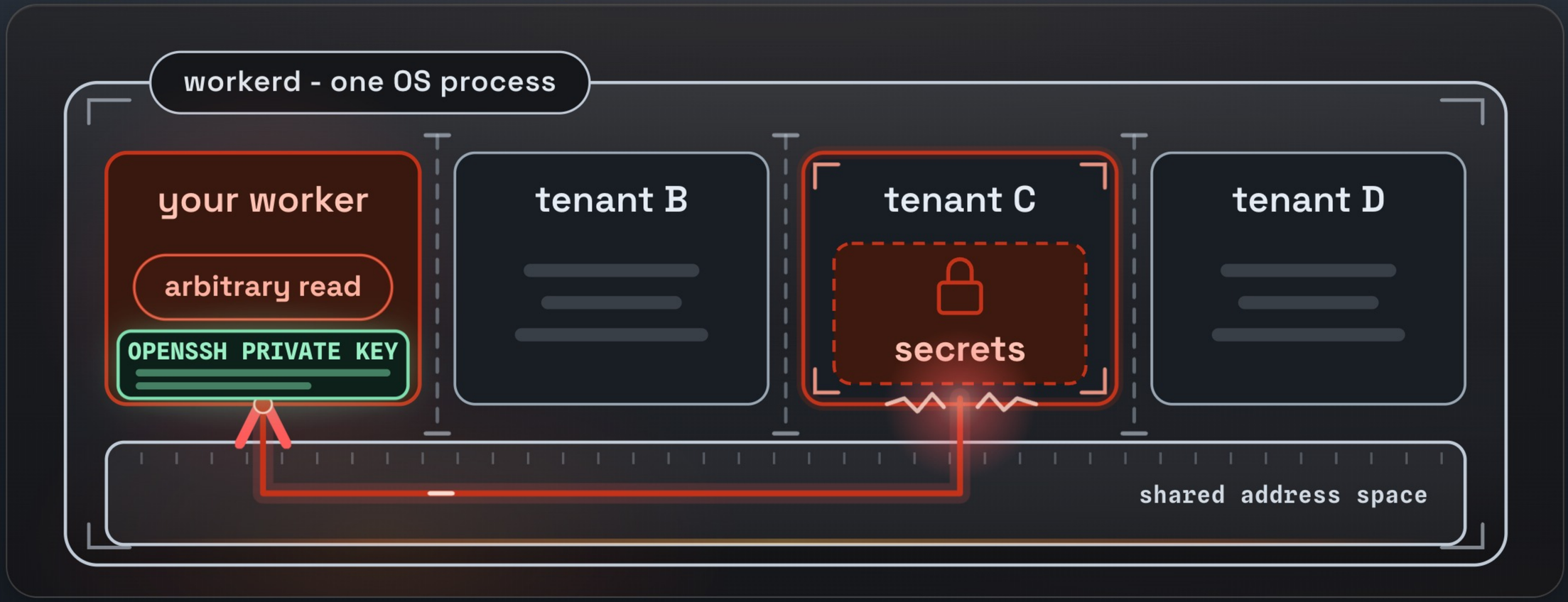
RESEARCH VULNS EXPLOITS CLOSING

03

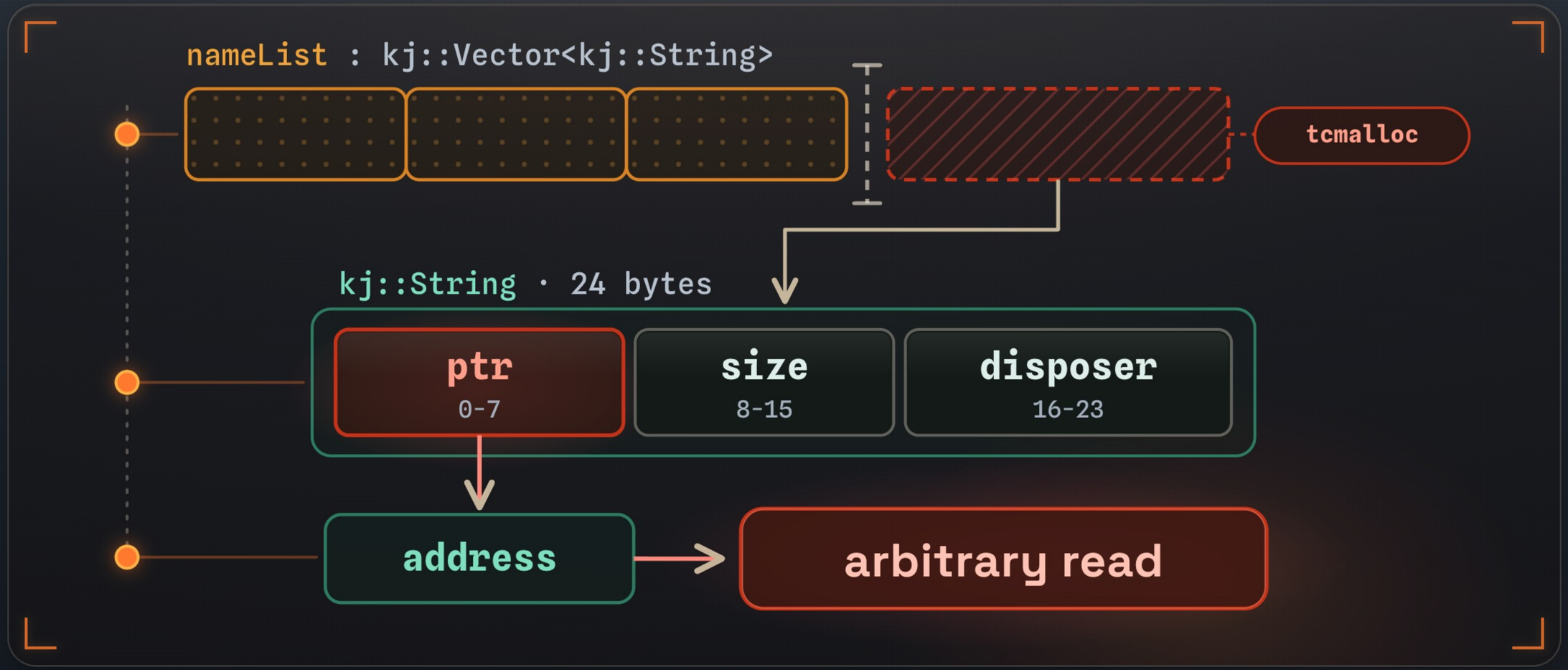
Exploits & Demos

from one out-of-bounds read to reading another tenant's secrets

Many tenants, one process



The primitive



The strategy

control a fake `kj::String { ptr, size, disposer }` → read the bytes at **ptr**

GOAL sweep the shared heap for another tenant's secret

1

Control the `ptr`

2

Make it *repeatable*

3

Break ASLR

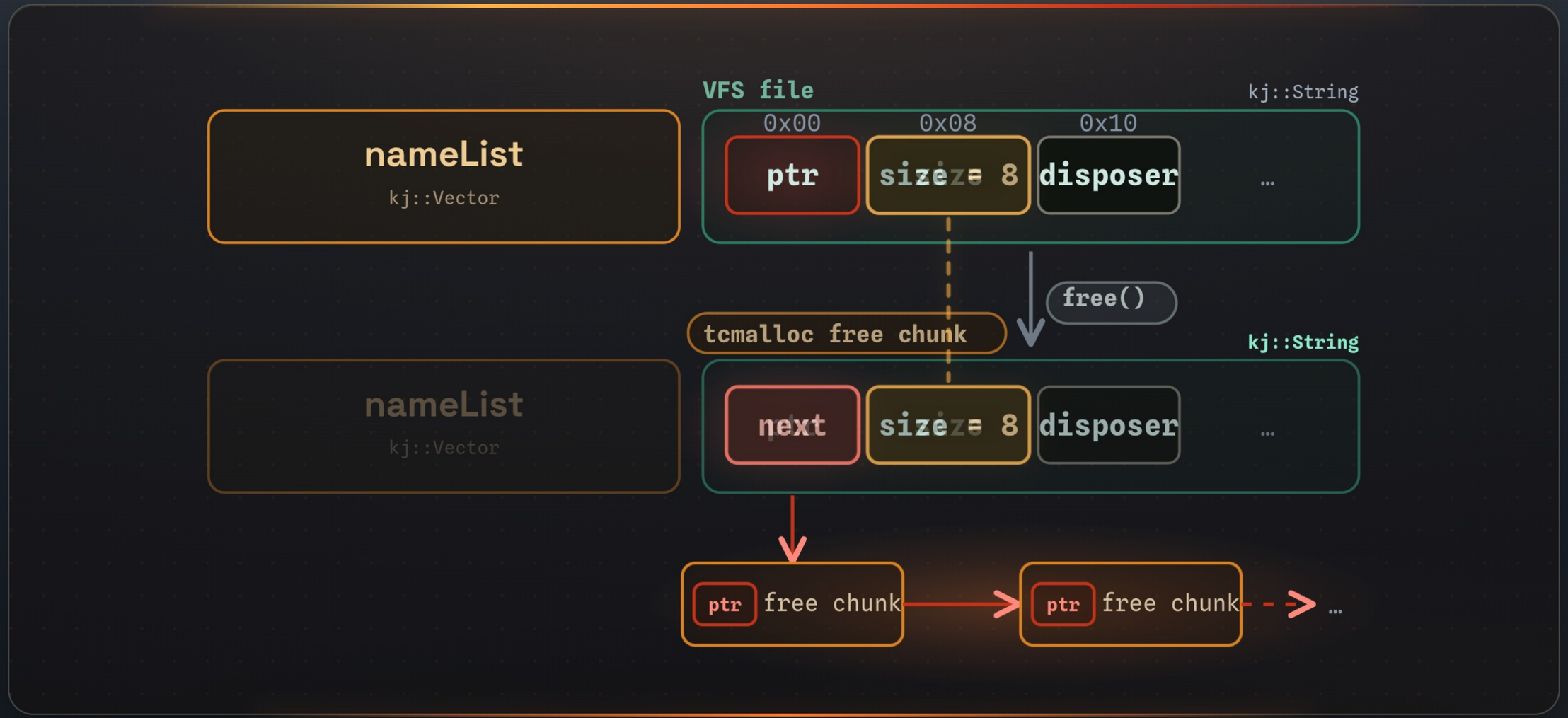
** No objects inside the cage were harmed in the making of this exploit.*

2 A repeatable read

- **VFS** - Virtual File System
- **VFS** - workerd backend for `node:fs`

3 Breaking ASLR

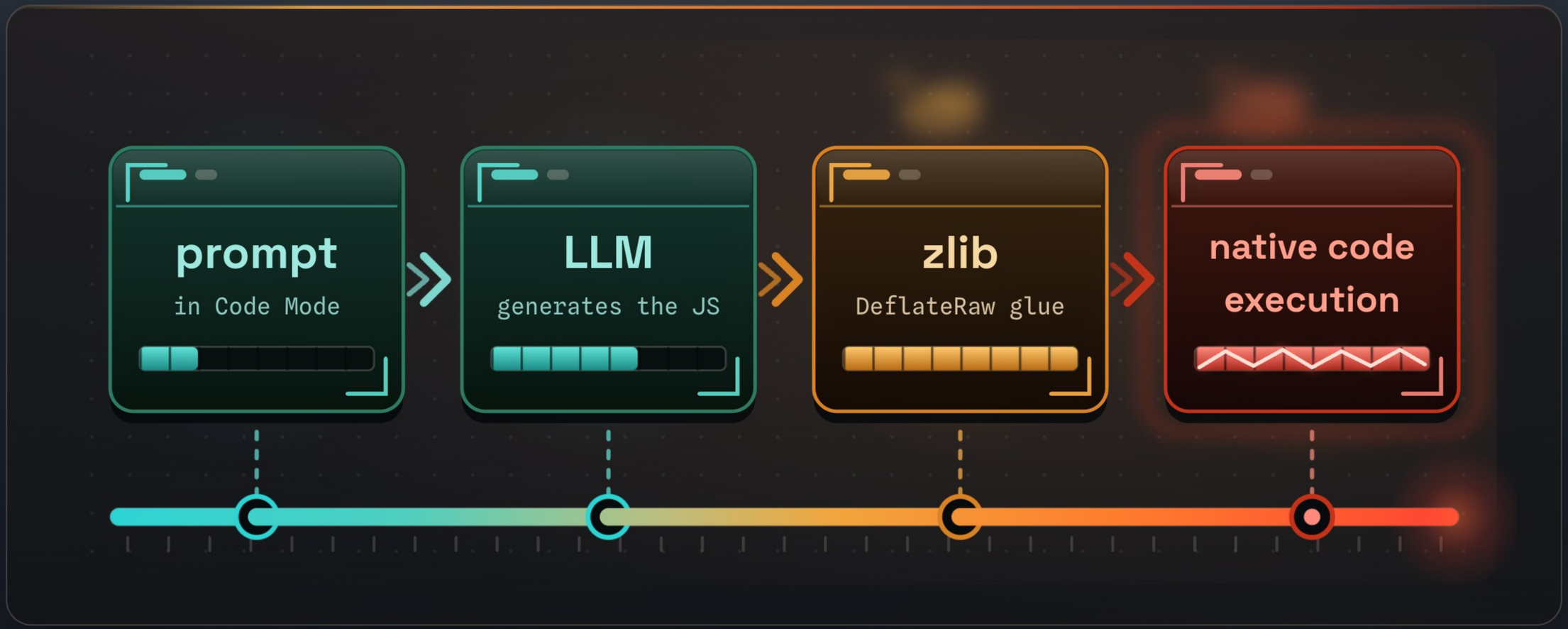
3 Breaking ASLR



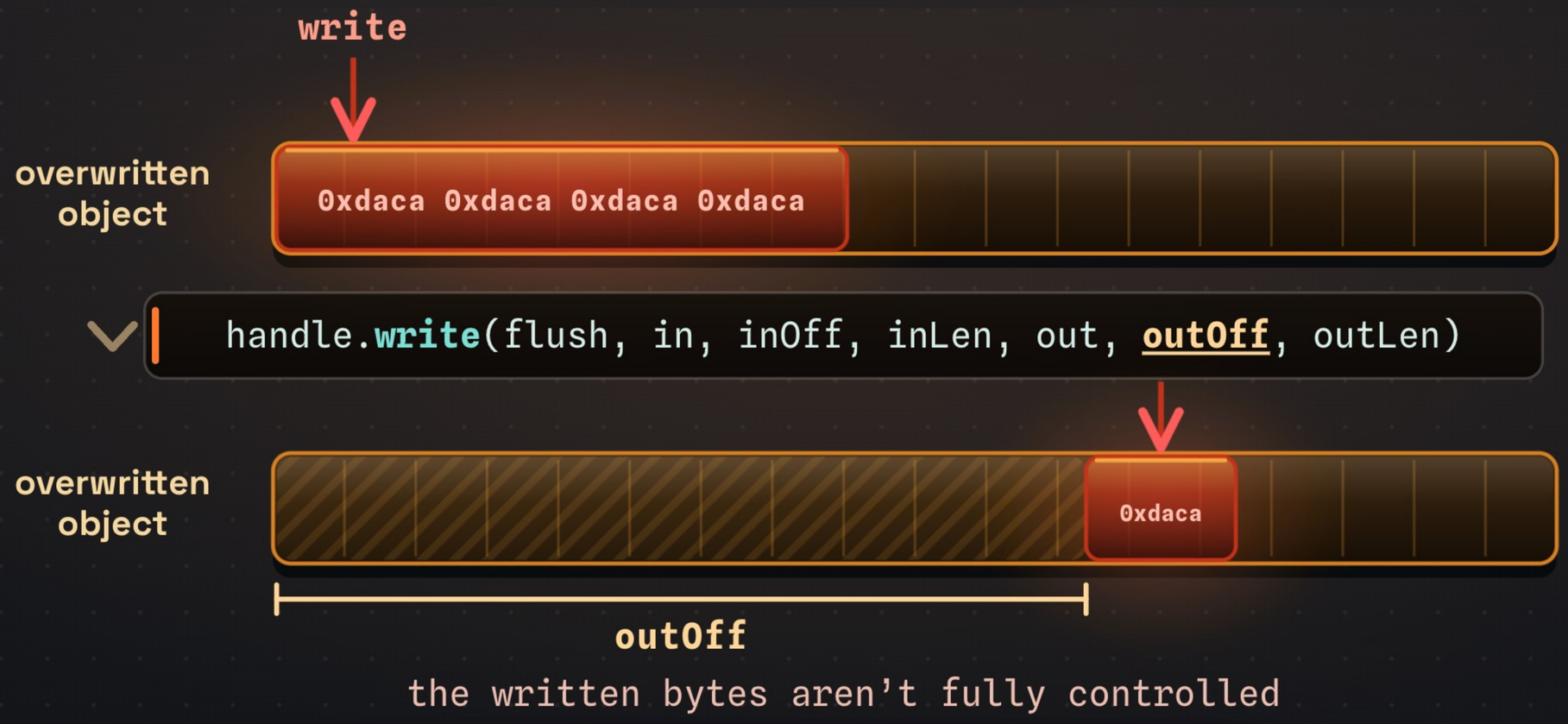
DEMO 1

```
victim % docker exec -it melt bash
root@3c8ac451c9e5:/exploit#
```

From a prompt to native code



Primitive improvement



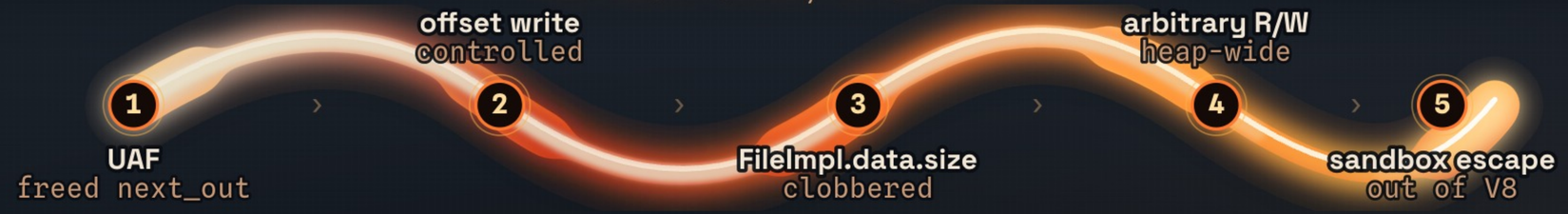
the written bytes aren't fully controlled

What we write: the FileImpl

- **FileImpl** - metadata of the file (VFS again)

From FileImpl to arbitrary R/W

THE WHOLE CHAIN, IN ONE LINE



CodeMode Testing App

Test MCP servers and chat with LLM

MCP Servers

Server Name | Server URL | Add

excel Remove

`http://host.docker.internal:9001/mcp`

Available Tools:

list_sheets | read_sheet

Chat with LLM

Reset Chat

Type your message... | Send

RESEARCH VULNS EXPLOITS CLOSING

04

Disclosure, and what it means

reported and fixed - then what actually holds in production

Coordinated, fixed, quiet

Feb 1, 2026

4 of 5 - zlib, HTMLRewriter, two URLPattern OOBs.

Mar 11, 2026

zlib + HTMLRewriter: Critical.

• 2× CRITICAL

Mar 12, 2026

KV SQL bypass.

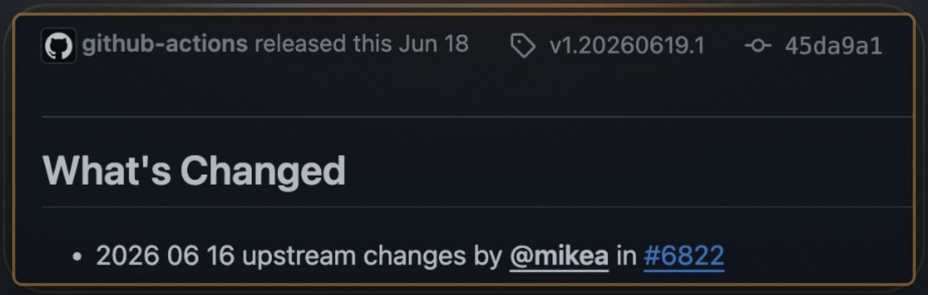
Jun 19, 2026

workerd v1.20260619.1. Managed Workers fixed first, on Cloudflare's side.

CVES ASSIGNED

0 0 0 0

by policy



• HackerOne: slow start

• Engineers: professional

• Bounty: paid

• Their own CNA: no CVE

We asked. Their answer: this project does not assign CVEs, by policy.

Two bugs. Two regions. Two honest verdicts.

URLPattern

● native heap · outside the cage + MPK

- → Native heap - the cage and the keys never covered it.
- → Ada-backed impl - the production default.

Cloudflare confirmed: triggered in production.

We never ran it there. A crash takes co-tenants with it.

node:zlib

● inside V8's sandbox

- → This one lands **inside** V8's sandbox - the mitigations do cover it.
- → Getting out needs deeper work, probably another bug.

Would not work as-is on production.

Critical UAF → arbitrary R/W*

** self-hosted.*

Very attainable. We stuck with the self-hosted demo. *Maybe next year.*

Context is the attack surface

GUARDRAIL vendor allow-listed · usd < 50000

WHO ASKED	ALL THE GUARDRAIL SEES	
the finance team	`pay_invoice("Nord Supply", usd=48000)`	✓ allowed
a line in the invoice	`pay_invoice("Nord Supply", usd=48000)`	✓ allowed

THE MISSING VARIABLE

- who is asking
- what this app is *for*
- what normal looks like *here*
- the whole trace so far

When Agentic Glue Melts

The cage protects the engine.
Mind the glue.

THANK YOU

Yarden Porat · Shahar Tal
Check Point Research

write-up